ProtNote

A multimodal method for protein–function annotation

Samir Char et al. — *Bioinformatics* 41(5), btaf170 (2025)

Central question: Can a protein function predictor accept a new function as text, without adding a trained output class?

Journal Club DOI: 10.1093/bioinformatics/btaf170

A fixed output vocabulary ages with the ontology

Conventional classifier

Protein → vector → scores for a fixed list of GO terms.

A term added after training has no output neuron. Updating its name does not change the learned output's meaning.

ProtNote

Protein + function description → one match score.

Changing the candidate text changes the query. New descriptions can be evaluated without retraining the model.

GO describes molecular function, biological process, and cellular component. Its terms, definitions, and graph relationships all change.

Main text, Introduction and Fig. 1; Supplement A, Fig. S1.

Three different meanings of “unseen”

Setting	Held out	What it tests
Supervised GO	Test sequences; function labels are known	Generalization within the trained label vocabulary
Novel-GO zero shot	Newly added sequences and post-2019 GO terms	Temporal transfer to unseen label descriptions
EC zero shot	EC annotation task and vocabulary	Transfer from GO text to enzyme descriptions

An unseen label is not necessarily a new biochemical capability. Its description can closely resemble a known GO function. Nor does a new sequence imply remote homology.

Main text, Sections 2.2 and 3.3; Supplement Table S3. Last distinction: interpretation.

A running example: query a sequence with a description

Illustrative inputs, not a biological annotation or model run

Sequence x: `MARGKKIGYSAPGKGTQA` (toy fragment)

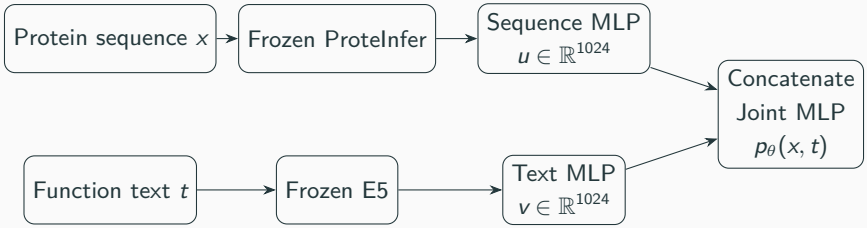
Candidate text t_1: "ATP binding"
Alternative text t_2: "DNA binding"

For each pair (x, t), ProtNote returns $p_\theta(x, t) \in [0, 1]$, a predicted annotation-match likelihood. It can assign multiple functions to one sequence.

The operation is to *score a supplied description*; it does not generate a function name. A real prediction requires a suitable full sequence and model evaluation. No scores are invented here.

Task formulation: Main text, Section 2.1 and Fig. 1B–D. Toy inputs: teaching example.

Two frozen encoders feed a learned pair classifier



Each projection: three hidden layers of 3072 units, then 1024 outputs. Joint MLP: three hidden layers of 3072, then one output. ReLU and batch normalization; no MLP bias parameters.

Main text, Section 2.3; Supplement Fig. S4. E5: multilingual sentence embedding model.

The score is learned from the pair, not cosine similarity

$$u = g_s(E_{\text{ProteInfer}}(x)), \qquad v = g_t(E_{\text{E5}}(t)), \qquad p_\theta(x, t) = \sigma(h([u; v])) .$$

1. Encode the toy sequence once; encode "ATP binding" and "DNA binding" separately.
2. Project each modality to 1024 dimensions; concatenate to a 2048-dimensional pair representation.
3. Run the joint MLP separately for each candidate text.

Frozen embeddings can be cached. But N proteins and L descriptions still require NL joint scores; this is not a dot-product retrieval model with a contrastive objective.

Main text, Sections 2.3 and 4. Sigmoid notation makes the binary probability output explicit.

Training labels include the GO ancestry closure

A protein's positive set contains its specific GO annotations *and their ancestors*; all other candidate GO terms are treated as negative.



Without closure, a correct parent could become a negative training target. Short term names and longer definitions supply two textual views of each label.

Interpretation: an unrecorded annotation is not experimental evidence of absence. Treating every missing label as negative creates possible false negatives.

Main text, Sections 2.1–2.3 and Fig. 1B. Missing-label caveat: interpretation.

Rare labels motivate both loss and sampling choices

For sequence i, term j, and binary label y_{ij}:

$$p_t = \begin{cases} p_{ij}, & y_{ij} = 1, \\ 1 - p_{ij}, & y_{ij} = 0, \end{cases} \qquad \mathcal{L}_{ij} = -(1 - p_t)^2 \log p_t.$$

Focal loss downweights easy examples; there is no α class-weight term.

$$\Pr(\text{sample sequence } i) \propto w_i = \sum_{j:y_{ij}=1} \frac{1}{\sqrt{f_j}},$$

where f_j is training frequency. A protein bearing rare annotations is sampled more often; the square root tempers extreme weights.

Main text, Section 2.1, Equations 1–2; Supplement Fig. S2: 80% of labels have at most 100 sequences.

Augmentation trades fitting for transfer

1. **Sequence:** each residue has probability 0.1 of substitution, sampled from conservative BLOSUM62 substitutions.
2. **Description:** sample name or definition with equal probability during training. At inference average the two probabilities, not the embeddings.
3. **Text embedding:** add independent uniform noise scaled by $20/\sqrt{1024}$ before its projection MLP.

Train the MLPs for 46 epochs on eight 32-GB V100 GPUs, batch size 256, Adam learning rate 3×10^{-4}, gradient clipping 1, mixed precision. Select by validation macro AP.

Main text, Sections 2.3 and 2.5; Supplement D. Both foundation encoders remain frozen.

The data design separates vocabulary and time

Dataset	Unique sequences	Labels	Construction
GO train	351,429	31,365	SwissProt random 80% split
GO validation	44,362	21,914	Random 10% split
GO test	43,575	22,026	Random 10% split
Novel GO, all nodes	56,212	614	New sequences and labels
Novel GO, leaf nodes	2,276	402	Leaf-only evaluation
EC test	21,870	2,534	ProteInfer EC test set

Training GO release: 1 July 2019. Novel GO uses May 2024 SwissProt and 17 June 2024 GO; sequences added after July 2019 and terms introduced after the training cutoff.

Main text, Section 2.2, Table 1. Unique-sequence counts from Table 1; Section 2.2 also reports deduplication and a 10,000-residue filter.

What average precision measures here

Per-label AP: rank proteins by a label's score, then integrate precision as recall increases. The paper uses TorchEval's rectangle-rule AUPRC implementation.

Macro AP

Average the per-label APs. Each label has equal weight; rare terms can strongly affect this metric.

Micro AP

Pool all sequence–label pairs. Frequent labels contribute many more positive examples.

These evaluate ranking over annotated candidates. They do not establish calibrated probabilities, a deployment threshold, or experimentally confirmed novel functions.

Main text, Section 2.5; Supplement C. Final sentence: interpretation of evaluation scope.

Supervised GO: a real macro-AP cost for flexibility

Model	Macro AP	Micro AP
ProtEx with exemplars	.7256	.9517
ProteInfer	.6418	.9032
ProtNote	**.6019**	**.9042**
BLAST top hit	.5249	.7899
ProtEx without exemplars	.3268	.8436

ProtNote nearly matches ProteInfer in micro AP but loses about 4.0 percentage points in macro AP. ProteInfer's molecular-function macro advantage is larger (.8059 versus .7077).

ProtEx uses extra self-supervised pretraining and BLAST-based retrieval. ProtNote/ProteInfer use five seeds; ProtEx/BLAST have one reported model. These are not resource-matched comparisons.

Main Fig. 2; Supplement Table S2. Table S2 values used consistently; rounded main-text numbers differ slightly.

Zero-shot baselines transfer the nearest known label

For a new term t_{new}:

$$j^* = \arg \max_{j \in \text{training labels}} \cos\big(E(t_{\text{new}}), E(t_j)\big), \qquad p(x, t_{\text{new}}) = p_{\text{ProteInfer}}(j^* \mid x).$$

The baseline's text encoder is E5 or BioGPT; it chooses a known label, then copies that label's ProteInfer score.

Example from Table S3: a C-end degron pathway is mapped to an N-end rule pathway. Similar words can conceal biologically important differences.

ProtNote learns how sequence and description interact. Beating these baselines establishes an advantage over *nearest-label score transfer*, not over every zero-shot annotation method.

Main text, Section 2.4; Supplement Table S3.

Novel GO: positive evidence with a reporting inconsistency

Main text, Section 3.3.1: reported macro AP values are .2139 (ProtNote), .1556 (E5), and .1236 (BioGPT), referring to Fig. 4B–C.

Supplement Table S1 / Fig. S5: the full ProtNote model's leaf-node zero-shot macro AP is $.2771 \pm .0146$ across five models.

These values do not agree. The supplement explicitly labels its ablation setting as novel GO leaf nodes. We preserve both reports rather than infer an undocumented reconciliation.

The qualitative conclusion is consistent: ProtNote exceeds both label-transfer baselines; Supplement Fig. S7 also reports leaf-plus-inferred-node and ontology-specific results.

Main Fig. 4B–C and Section 3.3.1; Supplement Figs. S5, S7 and Table S1.

EC: transfer to a new annotation vocabulary

Create one EC text by concatenating its class, subclass, sub-subclass, and specific enzyme descriptions with commas. Score it once; no short/long ensembling.

Model	Macro AP	Micro AP
ProtNote	**.3062**	**.3456**
E5 label transfer	.2058	.1107
BioGPT label transfer	.0961	.0714

ProtNote leads across all seven top-level EC classes; ligases are easier and translocases harder in these experiments.

EC labels were not training targets. Nevertheless GO training already describes enzyme chemistry: this is cross-vocabulary transfer, not proof of learning entirely new chemistry.

Main Fig. 4D–F and Section 3.3.2; Supplement Fig. S8, Table S3. Last distinction: interpretation.

Ablations: text regularization helps unseen descriptions

Variant	Supervised macro AP	GO zero-shot macro AP
Full ProtNote	.6019	.2771
No embedding noise	.6304	.2205
No description sampling	.6239	.2374
BioGPT text encoder	.6250	.2312
No weighted sampling	.5140	.2770

Noise and two-description training reduce supervised fit but improve transfer. Weighted sampling strongly helps supervised macro AP, with little change in this zero-shot summary.

Individual ablations in Table S1 do *not* surpass ProteInfer's .6418 macro AP, despite broader language in the main text. Full model: five seeds; ablations: three.

Supplement Table S1 and Fig. S5; main text Section 3.1. Zero shot here is the supplement's leaf-node setting.

Representation plots support organization, not mechanism

1. Joint pair embeddings separate true annotations from negatives under UMAP; positive pairs also group by GO ontology.
2. Sequence projection similarity correlates with annotation-set similarity: Pearson $r = .533$ in the sampled test subset.
3. Label–label similarities in text space and median protein space correlate: $r = .404$ (Supplement Fig. S11).

These are post hoc observations on frequent-label subsets. UMAP structure and correlations do not prove causal biological understanding, calibrated confidence, or performance on rare functions.

Main Fig. 3: 1,731 frequent annotations; Supplement Fig. S11: 1,502 labels. Final paragraph: interpretation.

Ontology drift and semantic distance both matter

Drift control: keep ProteInfer's original test sequences and vocabulary, but update their annotations to 2024. Macro AP falls by 11.67 percentage points; micro AP by 22.51 points.

This isolates annotation-version change from a change in test sequences, but does not isolate graph topology from every other annotation update.

Stratification: unseen terms more similar in text to known terms have higher AP (Fig. S13). Molecular function transfers better than cellular component; only 14% of positive training GO terms are cellular component.

Supplement A, Figs. S3, S7, S13; Main Section 3.3.1. Isolation caveat: interpretation.

What is established, and what remains open

Demonstrated

One sequence–text scoring model handles known GO labels, post-cutoff GO labels, and EC descriptions.

It improves on nearest-label transfer and retains strong supervised micro AP.

Unestablished

Remote-homology performance: the supervised split is random, and temporal novelty does not enforce sequence dissimilarity.

Free-form paraphrase robustness, truly new biology, experimental discovery, and probability calibration.

The protein encoder is a frozen, GO-supervised ProteInfer model; zero shot is relative to the training vocabulary, not absence of prior protein-function supervision.

Main Sections 2.2–2.3 and 4; distinctions and proposed evidence boundary: interpretation.

Discussion: where would this be useful?

1. **Ontology maintenance:** prioritize candidate proteins for a newly added term; use text similarity to flag extrapolations far from training descriptions.
2. **Stronger evaluation:** combine temporal and homology-controlled splits; stratify by text similarity and train-label frequency.
3. **Discovery test:** compare prespecified unseen-function queries with assays; include paraphrases, hard neighboring functions, and calibration checks.

Takeaway: replacing a fixed output class with a textual query enables useful label transfer. The key unresolved question is how far that transfer extends beyond familiar semantic neighborhoods.

Applications and next experiments: our interpretation, informed by Main Discussion and Supplement Fig. S13.

Sources and reproducibility boundary

Journal article

Char S, Corley N, Alamdari S, Yang KK, Amini AP. *ProtNote: a multimodal method for protein–function annotation*. Bioinformatics 41(5):btaf170 (2025).

doi.org/10.1093/bioinformatics/btaf170

Read for these notes

Full journal text via Europe PMC, PMC12054973; complete 12-page journal Supplementary Information (Sections A–E; Tables S1–S3; Figs. S1–S13).

Author resources

github.com/microsoft/protnote; Zenodo 13897920 (data, weights, computed metrics).

These notes reproduce reported numbers, not model runs. Dataset counts and GO zero-shot discrepancies are preserved as source qualifications. All diagrams are explanatory redrawings.